

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики
Кафедра программного обеспечения
и администрирования информационных систем

Реализация веб-приложение для молочной компании

Курсовая работа
Направление 02.03.03 Математическое обеспечение
и администрирование информационных систем

Зав. кафедрой _____

к. ф.-м. н., доц. Барановский Е. С.

Обучающийся _____

3 курс, 91 группа, Миа Музамил Ахмад

Руководитель _____

доц. Ефремов М. С.

Воронеж 2026

Аннотация

Данная работа посвящена разработке информационной системы DairyProduction, предназначенной для автоматизации процессов молочного предприятия. Система обеспечивает управление категориями продукции, товарами, клиентами, поставщиками, заказами, оборудованием и другими сущностями, необходимыми для функционирования предприятия.

В работе рассмотрены архитектура системы, структура базы данных, средства реализации, а также проведён анализ задачи и обоснование выбора технологий. Реализована серверная часть на платформе .NET с использованием принципов чистой архитектуры, ORM-фреймворка Entity Framework Core и реляционной СУБД PostgreSQL.

Проведено модульное тестирование сервисного слоя. Результатом работы является готовая к расширению и внедрению информационная система.

Содержание

Аннотация	2
Содержание	3
Введение	5
Постановка задачи	6
Средства реализации	7
1. Анализ задачи	8
1.1. Общий анализ задачи	8
1.2 Предполагаемые структуры данных	9
1.3 Предполагаемая структура программы	9
2. Теоретическая часть	11
2.1 Основные понятия информационных систем	11
2.2 Реляционная модель данных	11
2.2.1 Определение	11
2.2.2 Связи между таблицами	12
2.3 ER-диаграмма базы данных	13
2.4 Объектно-ориентированное программирование (ООП)	14
2.4.1 Определение	14
2.5 ORM и концепция отображения объектов на таблицы	14
2.5.1 Определение ORM	14
2.5.2 Entity Framework Core	14
2.6 Архитектурные принципы построения программных систем	15
2.6.1 Чистая архитектура (Clean Architecture)	15
2.7 Модульное тестирование	15

2.7.1 Определение	15
2.7.2 Используемые инструменты.....	15
2.7.3 Цель тестирования	15
Заключение	16
Список литературы	17

Введение

Современные предприятия молочной промышленности сталкиваются с необходимостью обработки больших объёмов данных: учёт продукции, управление заказами, взаимодействие с клиентами и поставщиками, контроль оборудования и логистики. Выполнение этих процессов вручную приводит к ошибкам, снижению эффективности и отсутствию единой информационной среды.

Целью данной работы является разработка программной системы, которая автоматизирует ключевые процессы молочного предприятия. В ходе работы была создана серверная часть информационной системы DairyProduction, включающая модель данных, бизнес-логику, репозитории, сервисы и модульные тесты. Система построена на современных технологиях .NET и соответствует требованиям расширяемости, надёжности и структурированности.

Постановка задачи

Необходимо разработать информационную систему, обеспечивающую:

1. Хранение данных о категориях продукции, товарах, клиентах, поставщиках, заказах, оборудовании, сотрудниках и других сущностях.
2. Обработку данных через сервисный слой с использованием DTO-моделей.
3. Выполнение CRUD-операций для всех сущностей.
4. Автоматическое управление миграциями базы данных.
5. Модульное тестирование бизнес-логики.
6. Чёткую архитектуру, разделяющую слои данных, логики и интерфейса.
7. Возможность дальнейшего расширения и интеграции с веб-интерфейсом.

Средства реализации

В проекте использованы следующие технологии:

1. Язык программирования:
 - a. C# (.NET 9)
2. Архитектурный подход:
 - a. Чистая архитектура (Clean Architecture)
 - b. Разделение на слои: Core, Infrastructure, Services, Web, Tests
3. Работа с базой данных:
 - a. PostgreSQL
 - b. Entity Framework Core 9
 - c. Миграции, DbContext, Fluent API
4. Тестирование:
 - a. xUnit
 - b. Moq
 - c. Модульные тесты сервисного слоя
5. Среда разработки:
 - a. Visual Studio 2022

1. Анализ задачи

Молочное предприятие ежедневно выполняет множество операций:

- учёт продукции и категорий;
- формирование заказов;
- взаимодействие с клиентами;
- работа с поставщиками;
- контроль оборудования;
- управление сотрудниками и отделами.

Для эффективной работы требуется единая система, которая:

- хранит данные в структурированном виде;
- обеспечивает быстрый доступ к информации;
- минимизирует ошибки;
- автоматизирует рутинные операции.

1.1. Общий анализ задачи

Система должна решать следующие задачи:

1. Информационные задачи:

- а. хранение данных в реляционной базе;
- б. обеспечение связей между сущностями;
- с. предоставление удобных моделей данных.

2. Функциональные задачи:

- а. создание, редактирование, удаление и просмотр данных;
- б. обработка заказов;
- с. расчёт стоимости;
- д. управление статусами.

3. Технические задачи:

- а. обеспечение надёжности;
- б. структурирование кода;
- с. тестирование бизнес-логики;

d. поддержка миграций.

1.2 Предполагаемые структуры данных

База данных включает 14 таблиц, среди которых:

1. Основные сущности:

- Category;
- Product;
- Client;
- ClientType;
- Supplier;
- Orders;
- OrderItem;
- Equipment;
- CollectionCenter;
- Department;
- Position;
- Employee;
- OrderStatus;
- Status.

2. Связи:

- Один-ко-многим (Category → Product, ClientType → Client);
- Многим-ко-одному (OrderItem → Product);
- Один-к-одному (Order → OrderStatus);
- Композитные связи (Order → OrderItem)

Все сущности представлены в виде классов в проекте Dairy.Core.

1.3 Предполагаемая структура программы

Проект разделён на слои:

1. Dairy.Core

- a. Сущности (Entity);

- b. Интерфейсы репозиториев

2. Dairy.Infrastructure

- a. DbContext;
- b. Репозитории;
- c. Миграции;
- d. Конфигурации таблиц.

3. Dairy.Services

- a. DTO-модели;
- b. Сервисы;
- c. Бизнес-логика.

4. Dairy.Web

- a. Веб-интерфейс (при необходимости);
- b. Razor Pages / MVC

5. Dairy.Tests

- a. Модульные тесты
- b. Mock-объекты
- c. Проверка логики сервисов

Такое разделение обеспечивает гибкость, расширяемость и удобство сопровождения.

2. Теоретическая часть

2.1 Основные понятия информационных систем

Информационная система — это совокупность программных и аппаратных средств, предназначенных для сбора, хранения, обработки и передачи данных. В контексте предприятий информационные системы позволяют автоматизировать бизнес-процессы, уменьшить количество ошибок и обеспечить доступ к актуальной информации.

Информационные системы строятся на основе нескольких ключевых теоретических концепций:

- структурированное хранение данных;
- модели данных;
- архитектурные принципы построения программ;
- методы обработки информации;
- принципы тестирования и контроля качества.

В данной работе рассматривается система, основанная на реляционной модели данных и объектно-ориентированном подходе.

2.2 Реляционная модель данных

2.2.1 Определение

Реляционная модель данных — это способ организации информации в виде таблиц (отношений), предложенный Эдгаром Коддом. Каждая таблица содержит строки (кортежи) и столбцы (атрибуты), а данные структурированы таким образом, чтобы обеспечивать целостность и логическую связанность.

Основные элементы реляционной модели:

- Таблица (relation) — множество записей одного типа;
- Атрибут (attribute) — поле таблицы, описывающее свойство сущности;
- Кортеж (tuple) — строка таблицы, представляющая конкретный объект;

- Первичный ключ (primary key) — уникальный идентификатор записи;
- Внешний ключ (foreign key) — ссылка на запись в другой таблице.

2.2.2 Связи между таблицами

В реляционных базах данных используются три типа связей:

- **Один-ко-многим:**
 - Category → Product;
 - Product → OrderItem;
 - Orders → OrderItem;
 - ClientType → Client;
 - Branch → Orders;
 - Department → Position;
 - Position → Employee;
 - Status → Employee;
 - Status → Supplier;
 - CollectionCenter → Equipment;
 - OrderStatus → Orders.
- **Многие-ко-одному** (обратные связи к «один-ко-многим»):
 - Product → Category;
 - OrderItem → Product.
- **Один-к-одному:**
 - Order → OrderStatus.
- **Многие-ко-многим:**
 - В системе DairyProduction нет прямых связей многие-ко-многим, но теоретически они могли бы быть.
 - например: Supplier ↔ Product (через промежуточную таблицу SupplierProduct)

Эти связи позволяют формировать целостную структуру данных предприятия.

2.3 ER-диаграмма базы данных

На рисунке 2.1 представлена ER-диаграмма информационной системы DairyProduction, отражающая основные сущности и связи между ними. Диаграмма построена на основе реляционной модели данных и включает 14 таблиц, связанных отношениями «один-ко-многим» и «многие-ко-одному».

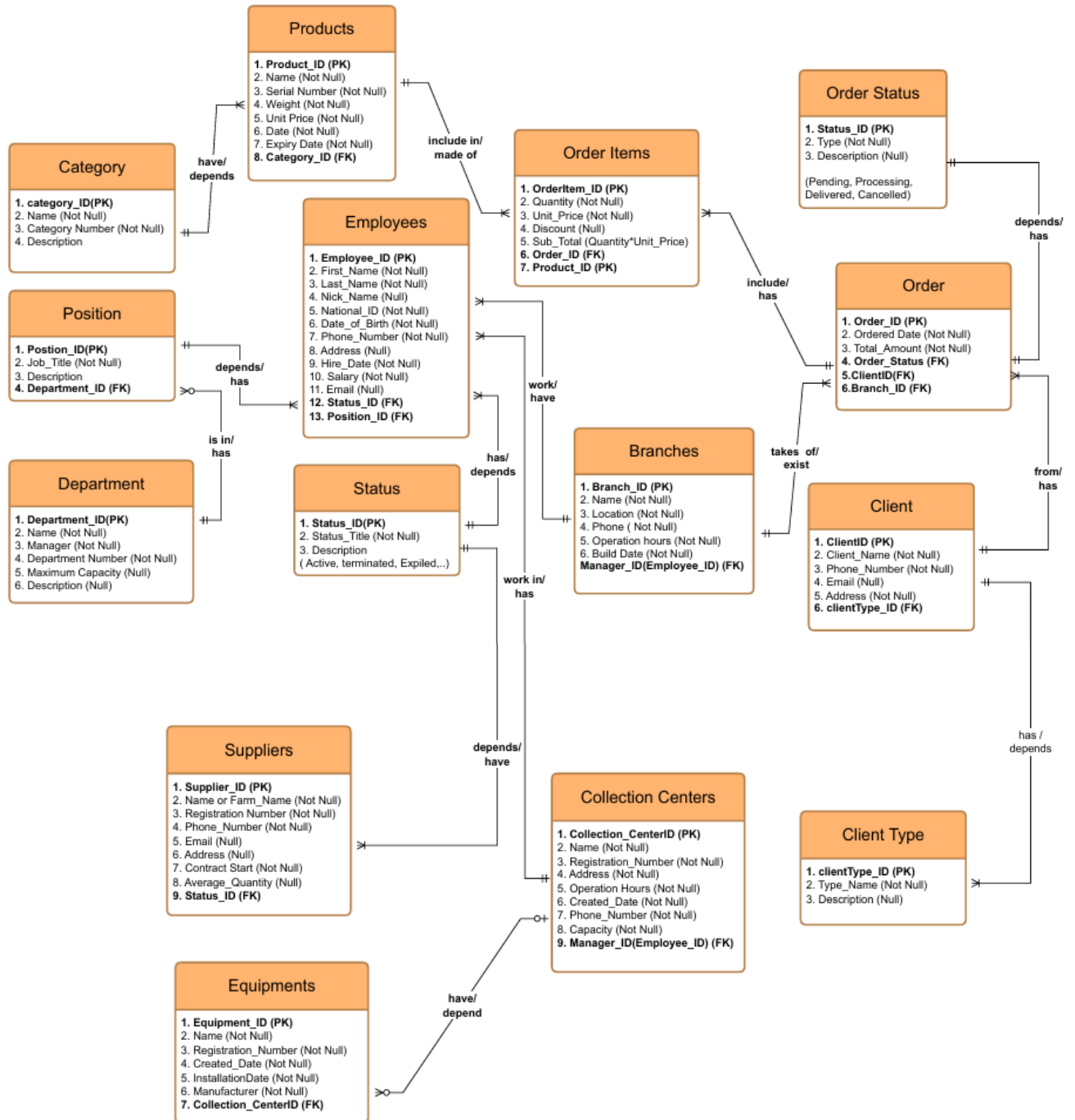


Рисунок 2.1 — ER-диаграмма базы данных DairyProduction.

2.4 Объектно-ориентированное программирование (ООП)

2.4.1 Определение

Объектно-ориентированное программирование — это парадигма, основанная на представлении программы как набора объектов, взаимодействующих между собой. Каждый объект имеет состояние (поля) и поведение (методы).

Основные принципы ООП:

1. Инкапсуляция — скрытие внутреннего состояния объекта.
2. Наследование — создание новых классов на основе существующих.
3. Полиморфизм — единый интерфейс для разных реализаций.
4. Абстракция — выделение значимых характеристик объекта.

В проекте DairyProduction ООП используется для описания сущностей предприятия, сервисов и репозиториев.

2.5 ORM и концепция отображения объектов на таблицы

2.5.1 Определение ORM

ORM (Object-Relational Mapping) — это технология, позволяющая автоматически преобразовывать объекты языка программирования в строки таблиц базы данных и обратно.

ORM решает проблему несоответствия между:

- объектной моделью программы;
- реляционной моделью базы данных.

2.5.2 Entity Framework Core

В проекте используется ORM-фреймворк Entity Framework Core, который обеспечивает:

- автоматическую генерацию SQL-запросов;
- создание миграций;

- строгую типизацию данных;
- удобную работу с репозиториями.

2.6 Архитектурные принципы построения программных систем

2.6.1 Чистая архитектура (Clean Architecture)

Чистая архитектура — это подход, основанный на разделении программы на независимые слои:

- Core — сущности и интерфейсы;
- Infrastructure — база данных и репозитории;
- Services — бизнес-логика;
- Web — пользовательский интерфейс.

Главная идея — зависимость направлена от внешних слоёв к внутренним, а не наоборот.

2.6.2 Принцип инверсии зависимостей

Высокоуровневые модули не должны зависеть от низкоуровневых. Оба должны зависеть от абстракций.

В проекте это реализовано через интерфейсы репозитория.

2.7 Модульное тестирование

2.7.1 Определение

Модульное тестирование — это метод проверки корректности работы отдельных компонентов программы в изоляции от остальных.

2.7.2 Используемые инструменты

- xUnit — фреймворк для тестирования;
- Moq — библиотека для создания имитаций зависимостей.

2.7.3 Цель тестирования

- проверка корректности бизнес-логики;
- выявление ошибок на ранних этапах;
- повышение надёжности системы.

Заключение

В ходе выполнения курсовой работы была разработана серверная часть информационной системы DairyProduction, предназначенной для автоматизации процессов молочного предприятия. В работе проведён анализ предметной области, сформирована реляционная модель данных, создана ER-диаграмма, определены основные сущности и связи между ними.

На основе принципов чистой архитектуры реализованы слои Core, Infrastructure и Services, обеспечивающие структурированность и расширяемость проекта. Для работы с базой данных использован ORM-фреймворк Entity Framework Core, что позволило автоматизировать создание миграций и обеспечить строгую типизацию данных. Бизнес-логика системы реализована в виде сервисов, использующих DTO-модели для безопасной передачи данных.

Особое внимание уделено модульному тестированию: с использованием фреймворков xUnit и Moq проверена корректность работы сервисного слоя, что повысило надёжность и качество программного обеспечения.

Разработанная система является функциональной основой для дальнейшего расширения: добавления веб-интерфейса, аналитических модулей, отчётности и интеграции с другими компонентами предприятия. Проект полностью соответствует поставленным задачам и демонстрирует практическое применение современных технологий разработки программных систем.

Список литературы

1. Microsoft Docs. Entity Framework Core Documentation. URL: <https://learn.microsoft.com/ef/> (дата обращения: 20.05.2026).
2. Microsoft Docs. ASP.NET Core Documentation. URL: <https://learn.microsoft.com/aspnet/core/> (дата обращения: 13.06.2026).
3. Beck K. Test Driven Development: By Example. Addison-Wesley, 2003.
4. Freeman S., Pryce N. Growing Object-Oriented Software, Guided by Tests. Addison-Wesley, 2009.
5. PostgreSQL Global Development Group. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата обращения: 22.05.2026).